

✓ Welcome to Colab!

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/dri

```
#!pip install langchain
#!pip install langchain_groq
#!pip install langchain_community
#!pip install langchain_text_splitter
#!pip install langchain_chroma
#!pip install langchain_groq
```

```
import os
#from langchain.document_loaders import UnstructuredFileLoader #old import not working
from langchain_community.document_loaders import UnstructuredFileLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_groq import ChatGroq
from langchain_classic.chains import RetrievalQA
```

```
/tmp/ipykernel_16030/1170151002.py:3: DeprecationWarning: `langchain-community` is being sunset and is
from langchain_community.document_loaders import UnstructuredFileLoader
```

```
os.environ["GROQ_API_KEY"]='gsk_YN6BYzsEP1GcuUkoHz5WGdyb3FYVcpWb1jQl2gDix3etamnsx9Z'
```

```
import requests
url="https://rbberger.github.io/assets/files/smr2574/OOPConcepts.pdf"
response = requests.get(url)
with open('OOPConcepts.pdf', 'wb') as f:
    f.write(response.content)
```

```
#!pip install unstructured[pdf]
```

```
loader = UnstructuredFileLoader("OOPConcepts.pdf")
```

```
/tmp/ipykernel_16030/743537071.py:1: LangChainDeprecationWarning: The class `UnstructuredFileLoader` wa
loader = UnstructuredFileLoader("OOPConcepts.pdf")
```

```
documents=loader.load()
documents
```

communication\n\nDesign Patterns: Main categories\n\nCreational Patterns\n\nStructural Patterns\n\nAbstract Factory • Builder • Factory Method • Prototype • Singleton\n\nAdapter • Bridge • Composite • Decorator • Façade • Flyweight • Proxy\n\nBehavioral Patterns\n\nChain of Responsibility • Command Interpreter • Iterator • • Mediator • Memento • Observer • State • Strategy • Template • Visitor\n\nCreational Patterns\n\nFactory Methods\n\nCreate objects without having to know specific language type • Circumvent limitations of constructors\n\nNo return result only exceptions\n\nConstrained naming\n\neg. can't have two constructors with same parameter types\n\nStatically bound creation\n\nthere is no dynamic binding for constructors, you have to know which type you want to instantiate\n\nNo virtual constructors\n\nFactory methods can range from very simple implementations to complex selection schemes\n\nCreational Patterns\n\nFactory Methods - Simple Example\n\nnclass IShape { public:\n\nIShape * createShape(const std::string & name) {\n\nvirtual void draw();\n\nif (name == "rectangle") {\n\n};\n\nreturn new Rectangle();\n\n}\n\nnclass Rectangle : IShape; class Circle : IShape; class Triangle : IShape;\n\nif (name == "circle") {\n\nreturn new Circle();\n\n}\n\nnclass ShapeFactory { public:\n\nif (name == "triangle") {\n\nIShape * createShape(const std::string & name);\n\nreturn new Triangle();\n\n}\n\n}\n\nreturn nullptr;\n\n}\n\n// parse user input ShapeFactory factory; string selectedShape = getUserInput();\n\n// create object at runtime IShape * new_object = factory.createShape(selectedShape);\n\nThis factory implementation is hard coded. But you can easily write an extensible factory.\n\nStructural Patterns\n\nAdapter\n\nUsed to make an object of one type compatible to another • Typical use case:\n\nYou defined your own types of objects with a certain interface • You want to use an external library to manipulate your objects • However the interface expected by library is different to the one you used • Instead of rewriting your code, you can create an Adapter class, which maps one interface to another.\n\nStructural Patterns\n\nAdapter - Example\n\nnclass ForceComputation { public:\n\nnclass LegacyClass { public:\n\nvirtual void compute_force(Vector3D & force);\n\nvirtual void compute_force(double * force);\n\n};\n\n}\n\nnclass ForceComputationAdapter : public ForceComputation {\n\nLegacyClass * legacy;\n\npublic:\n\nForceComputationAdapter(LegacyClass * src) : legacy(src) {\n\n}\n\nvirtual void compute_force(Vector3D & force) {\n\ndouble f[3]; legacy->compute_force(&f[0]); force.x = f[0]; force.y = f[1]; force.z = f[2];\n\n}\n\n}\n\n}\n\nBehavioral Patterns\n\nStrategy\n\nUsed to keep parts of a larger implementations replaceable • You define a common interface to do a certain task • Any class which implements that interface can be used in larger implementation\n\nAllows you to exchange object of that interface during runtime • Typical use case:\n\nDefine a common interface to get data • Interface can be implemented by classes which use files, databases, web services, etc.\n\nBehavioral Patterns\n\nStrategy - Example\n\nnclass IRandomNumberGenerator { public:\n\ndouble getNextDouble() = 0;\n\n}\n\nnclass DiceRoll : public IRandomNumberGenerator { public:\n\ndouble getNextDouble() {\n\n// guaranteed to be random, // determined with a fair dice roll return 4;\n\n}\n\n}\n\nnclass MyUncrackableEncryption {\n\nIRandomNumberGenerator * random;\n\n}\n\n}\n\n}\n\nvoid setRandomNumberGenerator(IRandomNumberGenerator * r) {\n\nrandom = r;\n\n}\n\n}\n\nvoid encrypt(char * data, size_t length) {\n\ndouble r = random->getNextDouble(); ...}\n\n}\n\n}\n\nMyUncrackableEncryption e; DiceRoll d;\n\nvoid decrypt(char * data, size_t length) {\n\nne.setRandomNumberGenerator(d);

```
text_splitter = CharacterTextSplitter(chunk_size=2000, chunk_overlap=400)
texts = text_splitter.split_documents(documents)
#
```

type

```
type(texts)
```

```
list
```

```
len(texts)
```

```
13
```

```
texts[6]
```

Document(metadata={'source': 'OOPConcepts.pdf'}, page_content='a->fullThrottle();\n\n// NEW! will call Boeing747::fullThrottle\n\n->fullThrottle();\n\n// NEW! will call Cessna206h::fullThrottle\n\nnc->fullThrottle();\n\nObjects of derived classes can be used as base class objects.\n\nPolymorphism allows us to modify the behavior of base classes and replacing the implementation of methods. • Polymorphic methods must be\n\ndeclared virtual (in C++)\n\nPolymorphism\n\n// store objects of different types in a // datastructure using its base class Airplane ** airplanes = new Airplane*[10]; ... for(int i = 0; i < 10; i++) {\n\n// will choose correct implementation // dynamically at runtime airplanes[i]->fullThrottle();\n\n}\n\n}\n\nvoid foo(Airplane & a) {\n\n// this function will work with // any class derived from Airplane\n\n}\n\nUse base class to implement general algorithms and data structures which work with any derived type\n\nDynamic dispatch determines the type of an object at runtime and executes the correct method\n\nAbstract classes\n\nnclass Airplane {\n\npublic:\n\nfloat longitude, latitude;\n\nfloat heading, speed;\n\nfloat altitude;\n\nvirtual void fullThrottle() = 0;\n\n};\n\nVirtual functions without implementation are called pure-virtual functions. • Classes containing pure- virtual functions can not be instantiated and are called abstract classes\n\nTheir behavior must be defined in derived classes\n\nIs-A vs. Has-A Relationship\n\nUse composition to manage

complexity • Use inheritance to extract common functionality • Use polymorphism to implement general algorithms which are independent of specific types\n\nComposition = Has-A Relationship E.g. A Boeing 747 has four jet engines.\n\nInheritance = Is-A Relationship:\n\nE.g. A Cessna is an airplane. So is a Boeing 747.\n\nIII. Best Practices\n\nRecommendations, Design Patterns\n\nKeep your interfaces simple, clean and consistent • Methods in a class should be as orthogonal as possible\n\n\uf0e0 avoid method that do almost the same\n\nMethods with the same name should do similar things • Use encapsulation')

```
embeddings = HuggingFaceEmbeddings()
```

```
/tmp/ipykernel_16030/3655315981.py:1: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.1.0 and will be removed in 0.2.0. Use `HuggingFaceHubEmbeddings` instead.
/tmp/ipykernel_16030/3655315981.py:1: LangChainDeprecationWarning: Default values for HuggingFaceEmbeddings are deprecated in LangChain 0.1.0 and will be removed in 0.2.0. Use `HuggingFaceHubEmbeddings` instead.
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits.
modules.json: 100% 349/349 [00:00<00:00, 30.9kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.47kB/s]

README.md: 100% 11.6k/11.6k [00:00<00:00, 942kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.90kB/s]

config.json: 100% 571/571 [00:00<00:00, 30.8kB/s]

model.safetensors: reconstructing file: 100% 438MB / 438MB, 39.4MB/s

model.safetensors: downloading bytes: 414MB, 36.1MB/s

Loading weights: 100% 199/199 [00:00<00:00, 5484.26it/s]

tokenizer_config.json: 100% 363/363 [00:00<00:00, 22.9kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 10.1MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 28.2MB/s]

special_tokens_map.json: 100% 239/239 [00:00<00:00, 22.4kB/s]

config.json: 100% 190/190 [00:00<00:00, 16.5kB/s]
```

```
persist_directory = 'vector_db'
```

```
vector = Chroma.from_documents(documents=texts, embedding=embeddings, persist_directory=persist_directory)
```

```
retriever = vector.as_retriever()
#retriever
```

```
#llm from groq
llm = ChatGroq(
    #groq_api_key=groq_api_key,
    model_name="qwen/qwen3.8-27b",
    temperature=0.7,
    max_tokens=512
)
```

```
llm
```

```
ChatGroq(metadata={'lc_versions': {'langchain-core': '1.6.1', 'langchain': '1.3.18'}}, client=
<groq.resources.chat.completions.Completions object at 0x79ce2a7be510>, async_client=
<groq.resources.chat.completions.AsyncCompletions object at 0x79cdd5ea52b0>, model_name='qwen/qwen3.8-27b', model_kwargs={}, groq_api_key=SecretStr('*****'), max_tokens=512)
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
```

```
query = "What are the functions mentioned in this PDF?"
response = qa_chain.invoke({"query": query})
print(response["result"])
```

Based on the provided text, the following functions (methods) are mentioned:

```
**Member Methods / Instance Methods:**
*   `Vector2D::length()`
*   `Boeing747::gearUp()`
*   `Boeing747::gearDown()`
*   `Airplane::fullThrottle()` (noted as virtual)
*   `Boeing747::fullThrottle()`
*   `Cessna206h::fullThrottle()`
*   `Engine` methods (implied by context like `fullThrottle`, though specific engine method names other
*   `Gear::up()`
*   `Gear::down()`

**Constructors:**
*   `Vector2D()` (Default constructor)
*   `Vector2D(double x1, double y1)` (Parameterized constructor)
*   `Point2D(double x1, double y1)`

**Static Methods:**
*   `Point2D::reset_count()`

**Destructors:**
*   `MyVector::~MyVector()`

**Helper/Global Functions:**
*   `sqrt()` (used inside `Vector2D::length`)
*   `foo(int param1, int param2)` (used in examples for stack/heap allocation)
```

```
print(response)
```

```
{'query': 'What are the functions mentioned in this PDF?', 'result': 'Based on the provided text, the f
```

```
print(response["source_documents"][0].metadata["source"])
```

```
OOPConcepts.pdf
```

```
# invoke the qa chain and get a response for user query
query = "Give me summary of all function from this pdf?"
response = qa_chain.invoke({"query": query})
print(response["result"])
print("***30")
print("Source Document:", response["source_documents"][0].metadata["source"])
```

Based on the provided text, here is a summary of all the functions (methods) mentioned, categorized by

```
### **1. `Vector2D` Class**
*   `length()`**: A member method that calculates and returns the Euclidean length of the vector usin
*   `Vector2D()`**: A default constructor that initializes member variables `x` and `y` to `0.0`.
*   `Vector2D(double x1, double y1)`**: A parameterized constructor that initializes member variables

### **2. `Point2D` Class**
*   `Point2D(double x1, double y1)`**: A constructor that initializes `x`, `y`, and assigns an `ident
*   `reset_count()`**: A **static method** that resets the static member variable `numPoints` to `0`.

### **3. `MyVector` Class**
*   `MyVector(int dim)`**: A constructor that dynamically allocates memory for a `double` array of si
*   `~MyVector()`**: A **destructor** that frees the dynamically allocated memory for the `data` arra

### **4. `Boeing747` Class**
*   `gearUp()`**: A public method that updates physics and internally calls `gear.up()` on the privat
*   `gearDown()`**: A public method that updates physics and internally calls `gear.down()` on the pr
*   `fullThrottle()`**: A **virtual method** (in the base `Airplane` class, overridden in `Boeing747`

### **5. `Cessna206h` Class**
*   `fullThrottle()`**: A **virtual method** (in the base `Airplane` class, overridden in `Cessna206h

### **6
```